


INSTITUTO FEDERAL CATARINENSE
CAMPUS RIO DO SUL
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

# COMPARAÇÃO DE DESEMPENHO ENTRE ESTRUTURAS DE DADOS

Carlos Fontanive
Diogo Lohn

Rio do Sul – SC
2026

# Sumário

**1 Introdução** **1**

**2 Objetivos** **1**

2.1 Objetivo geral . . . . . 1

2.2 Objetivos específicos . . . . . 1

**3 Estruturas implementadas** **1**

**4 Metodologia** **3**

4.1 Ambiente de execução . . . . . 3

4.2 Conjuntos de dados . . . . . 3

4.3 Operações avaliadas . . . . . 3

4.4 Protocolo de medição . . . . . 4

**5 Resultados e discussão** **4**

5.1 Visão geral . . . . . 4

5.2 Construção . . . . . 5

5.3 Busca por elementos presentes . . . . . 7

5.4 Busca por elementos ausentes . . . . . 8

5.5 Inserção . . . . . 9

5.6 Remoção . . . . . 10

5.7 Consulta por intervalo . . . . . 11

**6 Limitações do experimento** **12**

**7 Conclusão** **12**

**8 Arquivos produzidos** **13**

# 1 Introdução

Estruturas de dados oferecem diferentes formas de organizar, armazenar e recuperar informações. A escolha da estrutura influencia diretamente o tempo necessário para realizar buscas, inserções, remoções e consultas, especialmente quando a quantidade de elementos aumenta. Além disso, estruturas que possuem a mesma finalidade podem reagir de maneiras muito diferentes à ordem em que os dados são inseridos.

Este trabalho compara experimentalmente sete estruturas: array linear, array ordenado, lista duplamente encadeada, tabela hash, árvore binária de busca sem balanceamento, árvore AVL e árvore rubro-negra. As implementações foram submetidas aos mesmos conjuntos de dados e às mesmas operações. Foram avaliados três tamanhos de entrada e três ordenações, utilizando cinco repetições medidas para cada combinação.

A análise combina os tempos obtidos com o comportamento assintótico esperado. O objetivo não é apenas indicar qual implementação foi mais rápida, mas mostrar por que determinadas estruturas se destacaram em algumas operações e tiveram desempenho inferior em outras.

# 2 Objetivos

## 2.1 Objetivo geral

Comparar experimentalmente o desempenho de diferentes estruturas de dados na execução de operações de construção, busca, inserção, remoção e consulta por intervalo.

## 2.2 Objetivos específicos

- implementar sete estruturas sem utilizar coleções prontas para representar as estruturas comparadas;
- aplicar os mesmos dados e operações a todas as implementações;
- avaliar entradas crescentes, decrescentes e aleatórias;
- analisar o efeito do aumento de 5.000 para 100.000 elementos;
- reduzir a influência de oscilações utilizando aquecimento e mediana;
- relacionar os resultados experimentais às complexidades esperadas.

# 3 Estruturas implementadas

Todas as classes implementam a interface `EstruturaDados`. Isso permite que o mesmo programa de medição execute as operações sem conhecer os detalhes internos de

cada implementação.

```java
public interface EstruturaDados {
    boolean inserir(int valor);
    boolean buscar(int valor);
    boolean excluir(int valor);
    int contarIntervalo(int min, int max);
    int tam();
    String nome();
}
```

Tabela 1: Características das estruturas implementadas

| **Estrutura** | **Característica principal** |
|---|---|
| Array linear | Vetor não ordenado, com busca e consulta por intervalo realizadas de forma sequencial. |
| Array ordenado | Vetor mantido em ordem crescente, com busca binária e deslocamento de elementos durante inserções e remoções. |
| Lista duplamente encadeada | Conjunto de nós ligados por referências para o elemento anterior e o posterior. |
| Tabela hash | Encadeamento separado para colisões, capacidade inicial prima e redimensionamento quando o fator de carga ultrapassa 0,75. |
| Árvore binária de busca | Árvore sem balanceamento automático, sujeita a degenerar em uma cadeia para entradas ordenadas. |
| Árvore AVL | Árvore balanceada por alturas, com rotações após inserções e remoções. |
| Árvore rubro-negra | Árvore balanceada por regras de coloração e rotações. |

No array linear e na lista, a verificação de duplicidade torna a construção de $n$ elementos uma operação quadrática. No array ordenado, busca e consulta por intervalo se beneficiam da ordenação, mas inserções e remoções podem exigir o deslocamento de muitos elementos. A tabela hash oferece custo médio constante para operações exatas, porém não preserva a ordem dos valores. A árvore binária de busca pode apresentar custo médio logarítmico ou custo linear no pior caso. AVL e rubro-negra mantêm altura logarítmica por meio do balanceamento.

# 4 Metodologia

## 4.1 Ambiente de execução

Todos os testes foram executados no mesmo computador e na mesma máquina virtual Java. Durante cada execução completa foram mantidas as mesmas configurações.

Tabela 2: Ambiente utilizado nos experimentos

| **Componente** | **Especificação** |
|---|---|
| Processador | Intel Core i5-13420H de 13ª geração |
| Memória RAM | 24 GB |
| Sistema operacional | Arch Linux, kernel 7.2.11-arch1-1 |
| Java | OpenJDK 27.0.1.0.3, 64 bits |
| Máquina virtual | OpenJDK 64-Bit Server VM, modo misto e compartilhado |

## 4.2 Conjuntos de dados

Foram utilizados conjuntos com 5.000, 10.000 e 100.000 elementos. Os valores iniciais eram números pares distintos no intervalo de 2 até $2n$. Para cada tamanho foram preparadas entradas crescentes, decrescentes e aleatórias.

As entradas aleatórias e as operações utilizaram sementes fixas e diferentes: 42 para a ordem de entrada, 43 e 44 para buscas presentes e ausentes, 45 para inserções, 46 para remoções e 47 para intervalos. Essa separação mantém a reprodutibilidade sem criar correlação artificial entre a ordem inicial e a ordem das operações.

## 4.3 Operações avaliadas

Cada estrutura foi submetida às seguintes operações:

- construção com todos os elementos iniciais;
- 2.000 buscas por valores presentes;
- 2.000 buscas por valores ausentes;
- inserção de uma quantidade equivalente a 10% do tamanho inicial;
- remoção de uma quantidade equivalente a 10% do tamanho inicial;
- 200 consultas por intervalos inclusivos com amplitude 100.

Os valores presentes e removidos pertenciam ao conjunto inicial. Os valores ausentes e inseridos eram ímpares, enquanto o conjunto inicial continha apenas números pares. As inserções não continham repetições.

## 4.4 Protocolo de medição

Os dados foram gerados antes do início do cronômetro. Cada combinação de estrutura, tamanho, ordem e operação recebeu uma execução de aquecimento, descartada dos resultados, seguida de cinco execuções medidas. A estrutura foi reconstruída antes de cada repetição, evitando que uma medição reutilizasse o estado deixado pela anterior.

O tempo foi obtido com `System.nanoTime()`. Impressões e gravações em arquivo ocorreram somente depois do encerramento do intervalo medido. O programa também verificou a quantidade de operações bem-sucedidas, interrompendo a execução caso alguma estrutura produzisse resultado diferente do esperado.

Ao todo, foram registradas 1.890 medições brutas:

$$3 \text{ tamanhos} \times 3 \text{ ordens} \times 7 \text{ estruturas} \times 6 \text{ operações} \times 5 \text{ repetições} = 1.890.$$

Para cada uma das 378 combinações foi calculada a mediana do tempo total. Os gráficos utilizam escala logarítmica no eixo vertical porque os tempos variam de milésimos de milissegundo a dezenas de segundos.

# 5 Resultados e discussão

## 5.1 Visão geral

A Tabela 3 apresenta a estrutura mais rápida na entrada aleatória com 100.000 elementos. A tabela hash dominou as operações baseadas em uma chave exata. O array ordenado apresentou o melhor resultado na consulta por intervalo.

Tabela 3: Menores medianas para entrada aleatória com 100.000 elementos

| **Operação** | **Estrutura** | **Tempo total (ms)** |
|---|---|---|
| Construção | Tabela hash | 0,630 |
| Busca presente | Tabela hash | 0,061 |
| Busca ausente | Tabela hash | 0,036 |
| Inserção | Tabela hash | 2,134 |
| Remoção | Tabela hash | 0,232 |
| Consulta por intervalo | Array ordenado | 0,031 |

Considerando todas as nove combinações de tamanho e ordem, a tabela hash foi a mais rápida em todas as construções, buscas presentes, buscas ausentes e remoções. Ela também venceu sete das nove situações de inserção. O array ordenado foi o mais rápido nas nove consultas por intervalo.

## 5.2 Construção

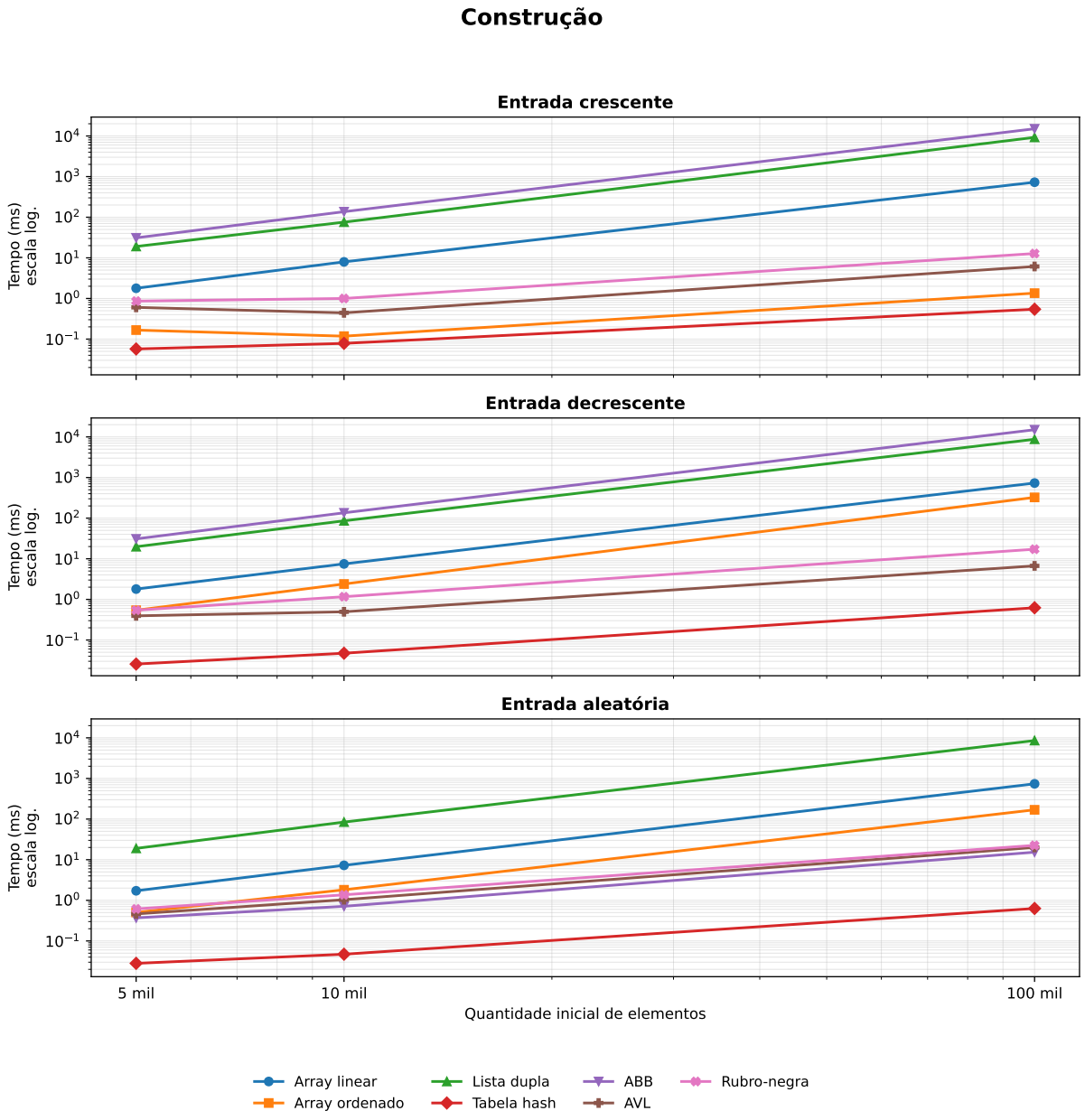



Figura 1: Mediana do tempo total de construção.

A tabela hash apresentou o menor tempo de construção nas nove combinações, chegando a 0,630 ms para 100.000 elementos aleatórios. AVL e rubro-negra também mantiveram tempos baixos e crescimento controlado. Na entrada aleatória com 100.000 elementos, seus tempos foram, respectivamente, 20,040 ms e 22,561 ms.

A ordem teve efeito decisivo sobre a árvore binária sem balanceamento. Sua construção levou aproximadamente 15.040 ms na ordem crescente e 15.024 ms na decrescente, contra 15,321 ms na entrada aleatória. Nas entradas ordenadas, cada novo nó foi colocado no extremo da árvore, que degenerou em uma cadeia. A diferença próxima de mil vezes

confirma o pior caso quadrático da estrutura.

O array ordenado também foi sensível à ordem. A construção crescente levou 1,352 ms porque os novos elementos foram acrescentados ao final. Na ordem decrescente foram necessários 324,400 ms, pois cada inserção deslocou os elementos existentes. A entrada aleatória apresentou resultado intermediário, 169,177 ms.

O array linear levou cerca de 727 a 737 ms nas três ordens. Seu custo é dominado pela busca sequencial usada para rejeitar duplicidades. A lista dupla apresentou os maiores tempos depois da árvore binária degenerada, entre 8,56 e 9,24 segundos para 100.000 elementos, devido às sucessivas travessias por nós.

## 5.3 Busca por elementos presentes

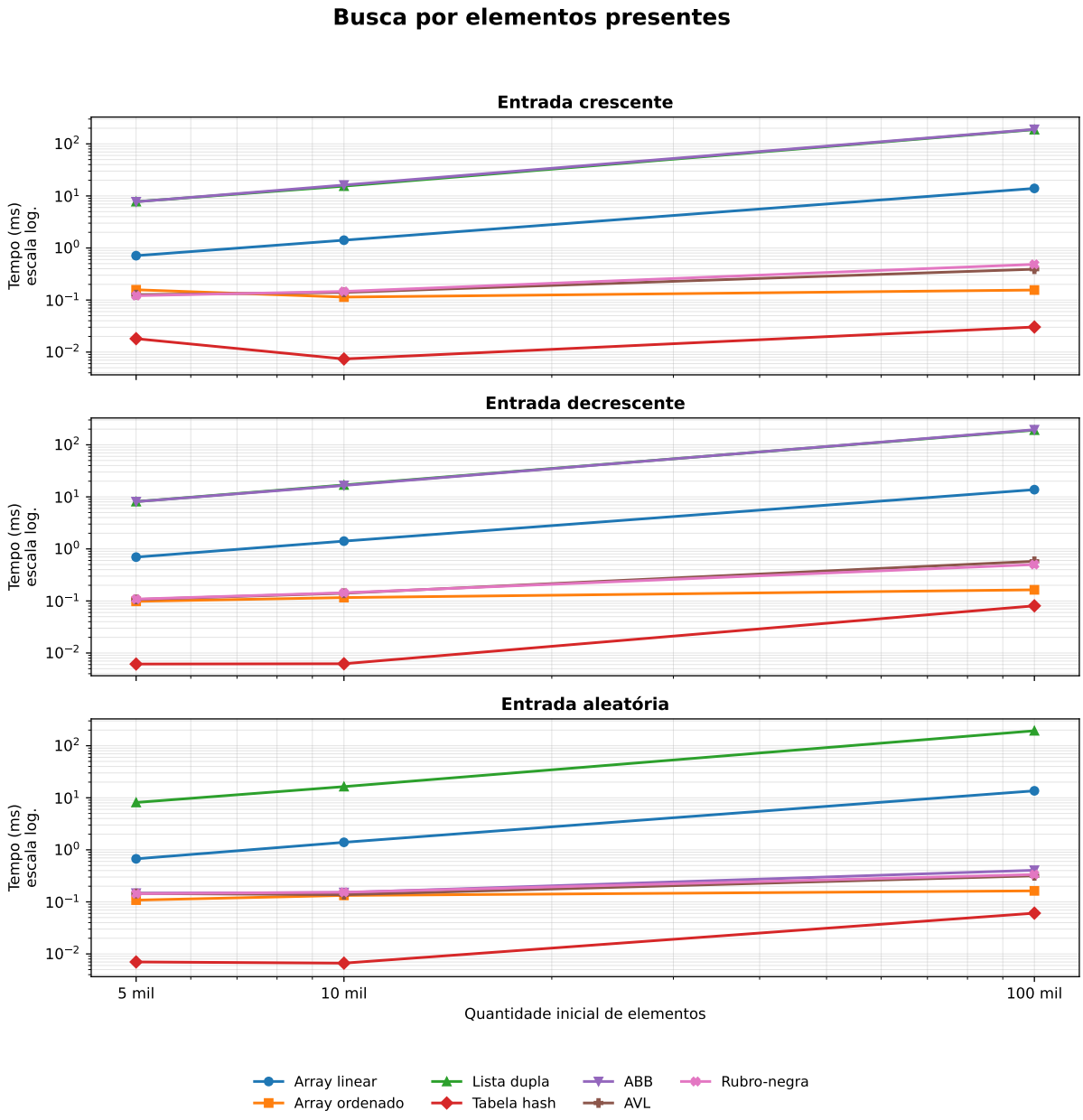



Figura 2: Mediana do tempo de busca por elementos presentes.

A tabela hash obteve os menores tempos, variando de 0,030 a 0,081 ms para as 2.000 buscas no conjunto de 100.000 elementos. O array ordenado ocupou a segunda posição, com aproximadamente 0,16 ms, resultado coerente com o uso de busca binária.

AVL e rubro-negra mantiveram tempos inferiores a 0,6 ms, independentemente da ordem inicial. A árvore binária sem balanceamento foi competitiva na entrada aleatória, com 0,405 ms, mas ultrapassou 190 ms nas entradas crescente e decrescente. Array linear e lista dupla realizaram busca sequencial e atingiram, respectivamente, cerca de 14 ms e 188 a 194 ms.

## 5.4 Busca por elementos ausentes

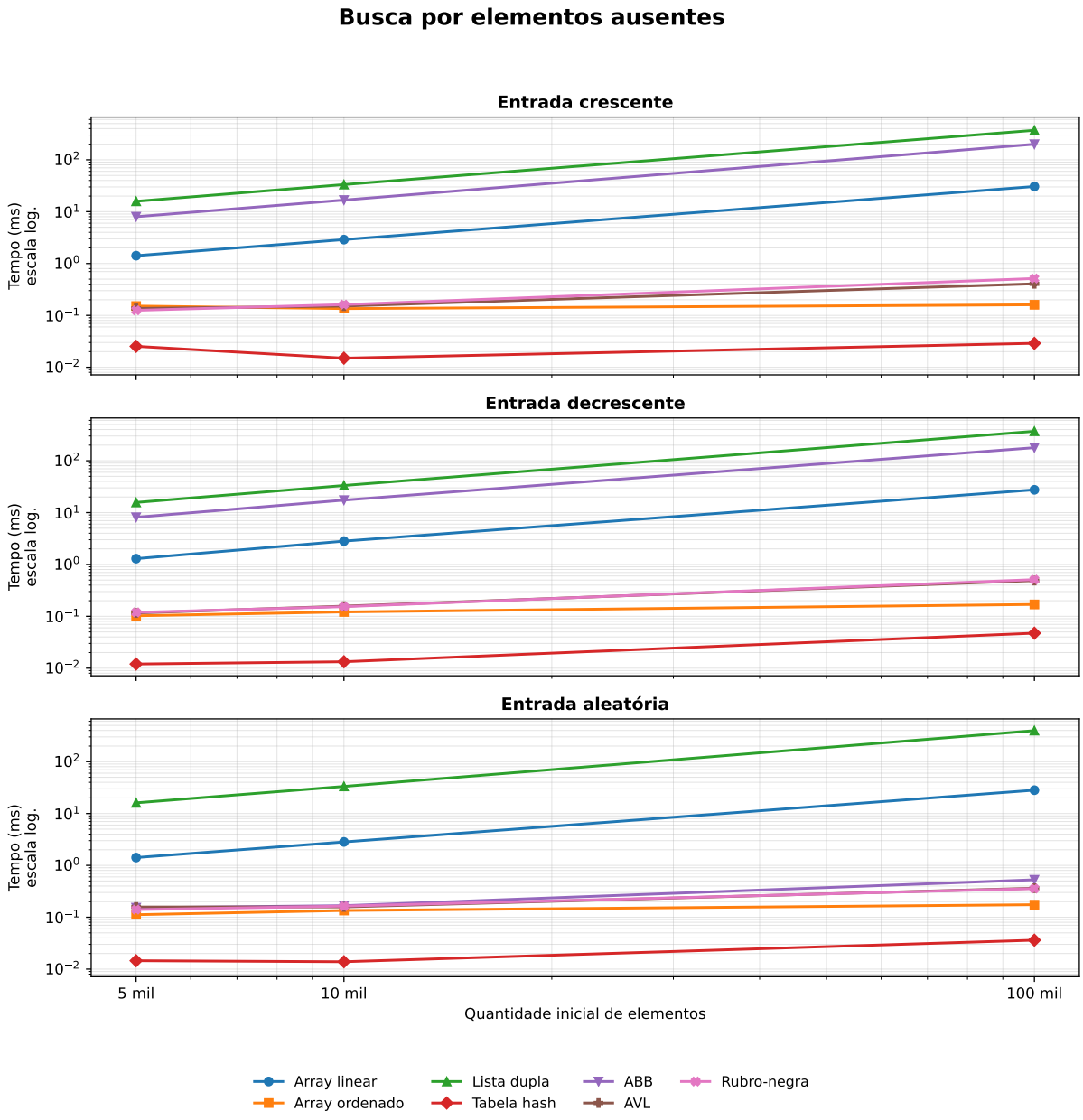



Figura 3: Mediana do tempo de busca por elementos ausentes.

As buscas ausentes ampliaram a diferença entre estruturas sequenciais e indexadas. Como o valor não era encontrado, array linear e lista precisavam percorrer todos os elementos. Com 100.000 valores, o array linear levou entre 27,49 e 30,58 ms, enquanto a lista ficou entre 370,11 e 396,56 ms.

A tabela hash permaneceu abaixo de 0,05 ms e o array ordenado abaixo de 0,18 ms. As árvores balanceadas permaneceram próximas de 0,36 a 0,51 ms. Na árvore binária sem balanceamento, a entrada aleatória resultou em 0,528 ms, mas as entradas ordenadas exigiram aproximadamente 179 a 200 ms.

## 5.5 Inserção

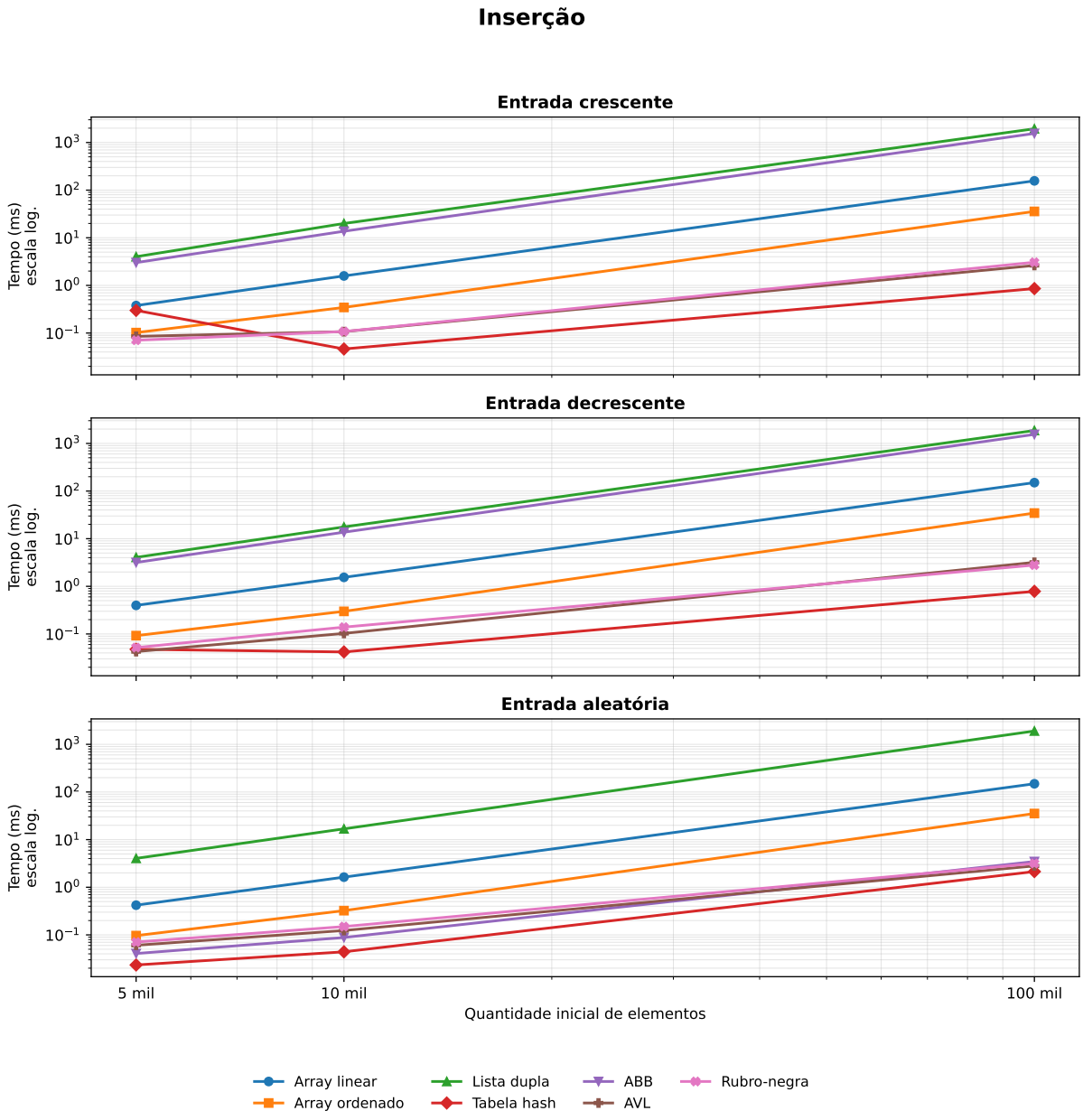



Figura 4: Mediana do tempo total das inserções adicionais.

Para 100.000 elementos iniciais, o teste inseriu 10.000 novos valores. A tabela hash apresentou os menores tempos, entre 0,785 e 2,134 ms. AVL e rubro-negra ficaram entre 2,62 e 3,19 ms, demonstrando que o custo do balanceamento ainda permite bom desempenho.

A árvore binária sem balanceamento levou apenas 3,468 ms quando havia sido construída aleatoriamente, mas aproximadamente 1.547 ms nas duas entradas ordenadas. O array ordenado ficou próximo de 35 ms e o array linear entre 149 e 157 ms. A lista dupla apresentou o pior resultado, entre 1.868 e 1.941 ms, principalmente devido à busca

necessária para rejeitar duplicidades.

## 5.6 Remoção

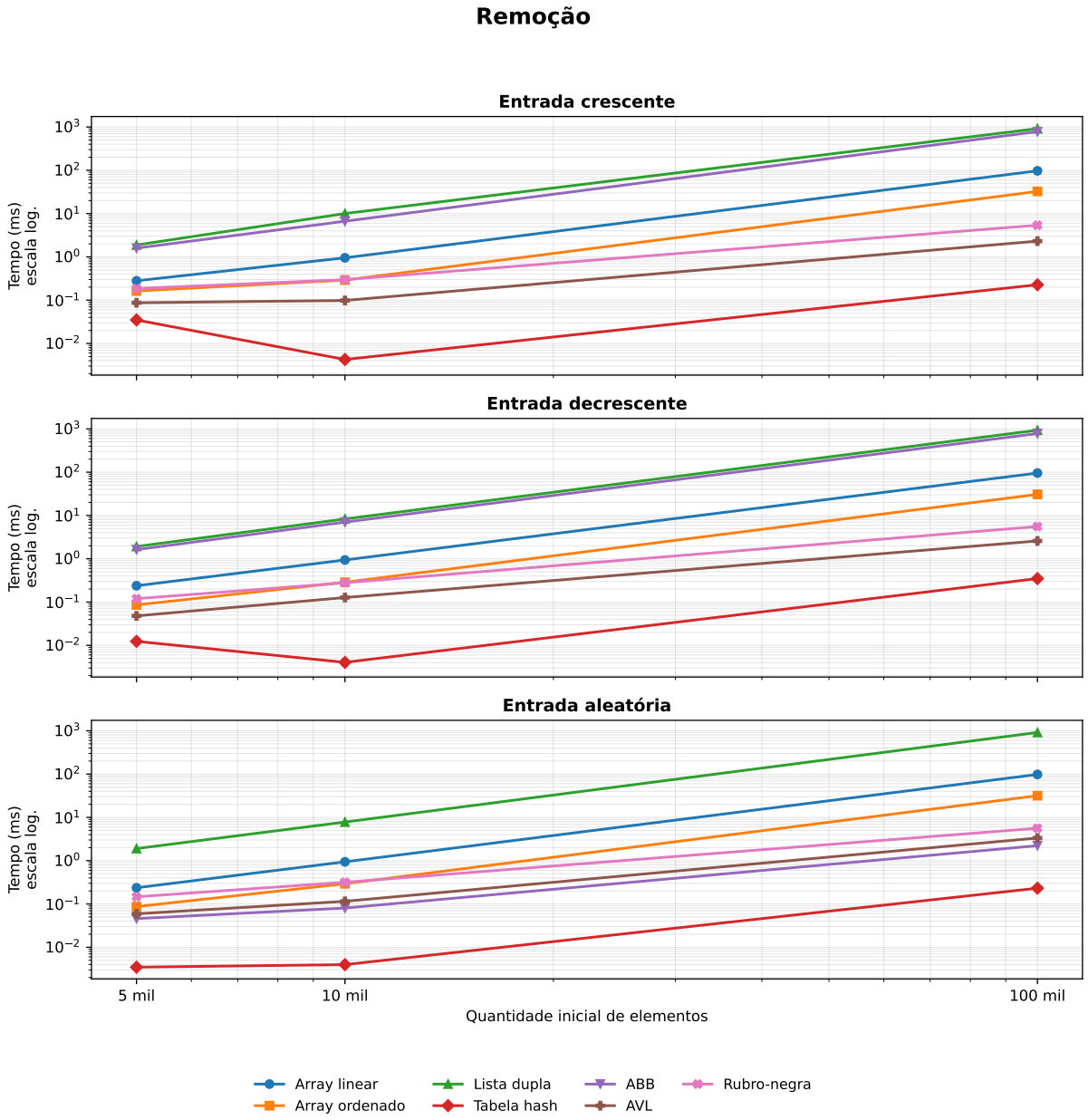



Figura 5: Mediana do tempo total das remoções.

Foram removidos 10.000 valores do conjunto com 100.000 elementos. A tabela hash foi novamente a mais rápida, com tempos entre 0,227 e 0,349 ms. AVL ficou entre 2,31 e 3,34 ms, enquanto a rubro-negra ficou entre 5,38 e 5,61 ms.

O array ordenado exigiu aproximadamente 31 a 33 ms e o array linear cerca de 96 a 98 ms. A lista dupla permaneceu próxima de 915 a 934 ms nas três ordens, mostrando que a correção das sementes eliminou a vantagem artificial que ocorreria se os elementos

fossem removidos exatamente na ordem da lista.

Na árvore binária sem balanceamento, a remoção aleatória levou 2,221 ms, mas as entradas ordenadas exigiram cerca de 782 ms. Novamente, o formato degenerado da árvore foi o principal fator de perda de desempenho.

## 5.7 Consulta por intervalo

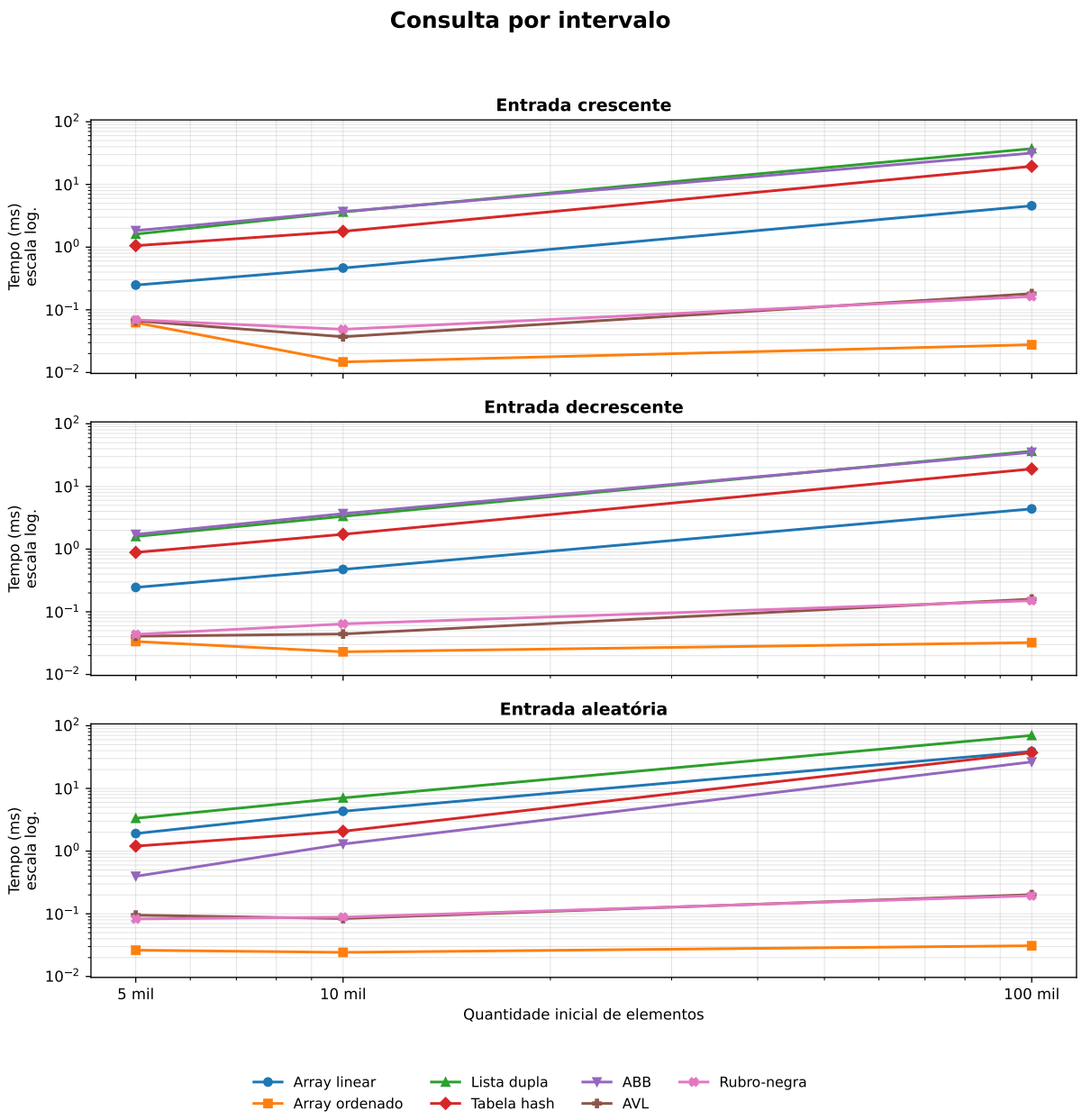



Figura 6: Mediana do tempo total das consultas por intervalo.

O array ordenado foi a estrutura mais rápida nas nove combinações. Para 200 consultas no conjunto de 100.000 elementos, os tempos ficaram entre 0,028 e 0,032 ms. A implementação localiza os limites do intervalo por busca binária, sem percorrer posições

que certamente estão fora da faixa.

AVL e rubro-negra também se destacaram, permanecendo abaixo de 0,21 ms. A ordenação permite descartar subárvores inteiras durante a consulta. A tabela hash, por não preservar ordem, precisou examinar os elementos e ficou entre 19,00 e 37,23 ms. A lista dupla variou entre 36,51 e 70,07 ms.

No array linear, as entradas crescente e decrescente ficaram próximas de 4,5 ms, enquanto a entrada aleatória chegou a 38,93 ms. A quantidade de comparações é linear em todos os casos, mas a disposição ordenada torna o resultado das comparações mais previsível para o processador. Na entrada aleatória, as alternâncias aumentam o custo de desvios condicionais.

# 6 Limitações do experimento

Os tempos representam uma máquina e uma implementação específicas. Resultados absolutos podem mudar com processador, versão da JVM, compilador, sistema operacional e atividade em segundo plano. Operações muito rápidas também são mais sensíveis ao escalonamento do sistema e à resolução prática da medição.

Uma execução de aquecimento reduz o efeito inicial da compilação JIT, mas não elimina todas as intervenções da máquina virtual, como coleta de lixo e novas otimizações. A mediana de cinco repetições diminui a influência de valores isolados. Portanto, as conclusões mais fortes são aquelas apoiadas por diferenças de ordem de grandeza e por tendências consistentes entre tamanhos e ordenações.

O experimento mede tempo, não consumo de memória. Além disso, as estruturas foram programadas para rejeitar valores duplicados. Essa decisão faz parte do custo observado, especialmente no array linear e na lista duplamente encadeada.

# 7 Conclusão

Os resultados mostram que não existe uma estrutura universalmente superior. A tabela hash foi a melhor escolha para construção e operações baseadas em uma chave exata, vencendo todas as buscas e remoções e a maioria das inserções. Sua principal limitação apareceu nas consultas por intervalo, pois ela não conserva a ordem dos valores.

O array ordenado foi especialmente eficiente em buscas e intervalos, mas pagou o custo de deslocamentos durante inserções, remoções e construções desfavoráveis. O array linear e a lista apresentaram crescimento aproximadamente quadrático nas operações que exigem busca repetida, tornando-se inadequados para grandes volumes quando não existe um índice adicional.

A árvore binária sem balanceamento demonstrou forte dependência da ordem de entrada. Com dados aleatórios, apresentou resultados competitivos; com dados ordenados,

degenerou em uma cadeia e ficou centenas ou milhares de vezes mais lenta. AVL e rubro-negra evitaram esse problema e mantiveram desempenho estável, confirmando a importância do balanceamento quando a ordem de chegada dos dados não pode ser controlada.

Assim, a escolha deve considerar as operações predominantes. Tabela hash é adequada para acesso exato, array ordenado para consultas em faixas e árvores balanceadas para conjuntos dinâmicos que precisam manter ordenação com bom desempenho no pior caso.

# 8 Arquivos produzidos

O programa gerou `resultados_brutos.csv`, contendo as cinco repetições de cada combinação, e `resumo.csv`, contendo a mediana do tempo total e o tempo mediano por operação. Os arquivos acompanham o projeto e permitem a reprodução dos gráficos apresentados neste relatório.

O código-fonte completo e os arquivos do experimento estão disponíveis no GitHub: https://github.com/GarrardFontanive/Comparacao_Estrutura_de_Dados.

# Referências


[1] CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

[2] SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Boston: Addison-Wesley, 2011.